

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	S.Sirivally	Reg. No.:	192472371
Section / Batch:	2024- CSE(AI)	Date:	12-08-2026

Instructions to Students

- Select/answer the assigned question. Each question carries **50 marks**. Duration 60mins
- Implement the solution using **Python**.
- Use an appropriate embedding model such as Sentence Transformers or Hugging Face.
- Use **FAISS and/or ChromaDB** as specified in the question.
- Demonstrate the complete pipeline:
Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis
- Execute the program and display meaningful outputs.
- Compare FAISS and ChromaDB where required.
- Provide appropriate graphs/tables for benchmark questions.
- Explain the architecture and design decisions.
- Submit working code and demonstrate the complete implementation.

Code of Conduct

I S.Sirivally / 192472371 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Sirivally.S____

SECTION A – VECTOR EMBEDDINGS & SEMANTIC SEARCH

13. FAISS Scalability Benchmark

Create datasets containing approximately:

100 documents

500 documents

1,000 documents

5,000 documents

10,000 documents

Build FAISS indexes for each dataset. Measure indexing time and average search latency. Plot the results and analyze the scalability of FAISS.

Problem Statement

Create datasets containing approximately 100, 500, 1,000, 5,000, and 10,000 documents. Build FAISS indexes for each dataset. Measure indexing time and average search latency. Plot the results and analyze the scalability of FAISS.

Aim

To evaluate the scalability and performance of the FAISS vector database by measuring indexing time and average semantic-search latency for different dataset sizes.

Objectives

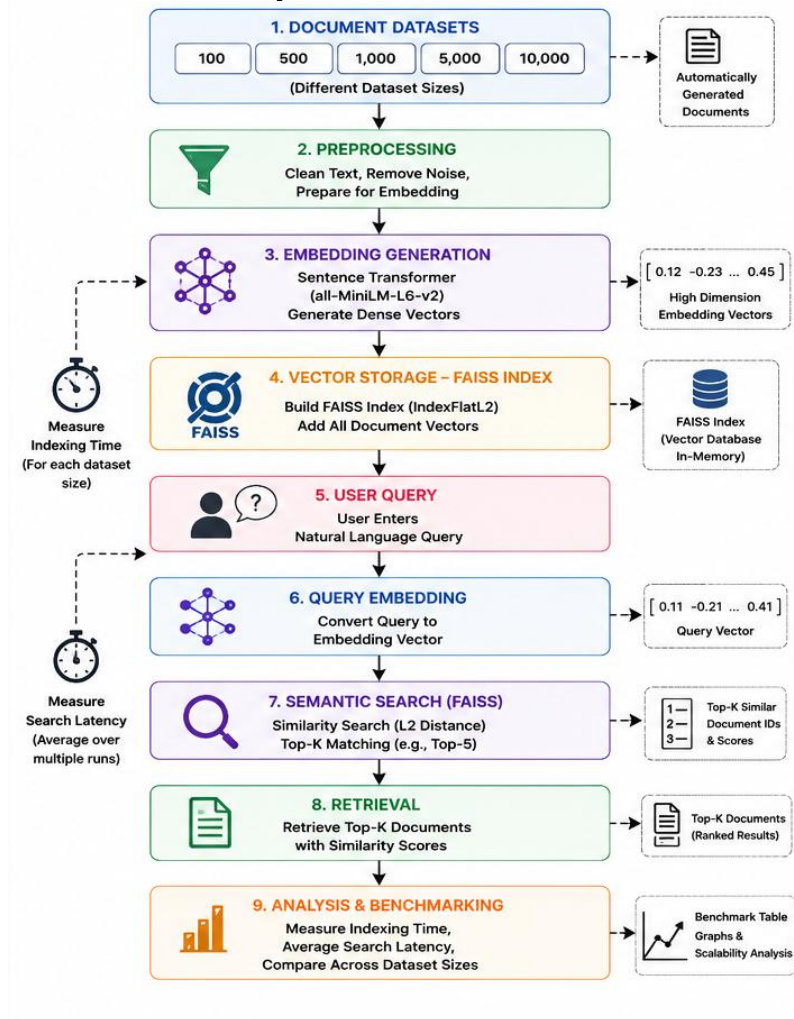
1. To create datasets containing 100, 500, 1,000, 5,000, and 10,000 documents.
2. To preprocess the documents.
3. To generate document embeddings using Sentence Transformers.
4. To build FAISS indexes for each dataset.
5. To measure FAISS indexing time.
6. To perform semantic similarity searches.
7. To calculate average search latency.
8. To compare the results for different dataset sizes.
9. To generate suitable graphs.
10. To analyze the scalability of FAISS.

Software and Technologies Used

Technology	Purpose
Python	Program implementation
Sentence Transformers	Generate document embeddings
all-MiniLM-L6-v2	Embedding model
FAISS	Vector indexing and similarity search
NumPy	Numerical processing
Pandas	Benchmark table
Matplotlib	Graph generation

System Architecture

The FAISS scalability benchmark follows the architecture shown below:



Architecture Explanation

The system consists of the following components:

1. Document Dataset:

The system generates datasets containing 100, 500, 1,000, 5,000, and 10,000 documents.

2. Preprocessing:

The generated text documents are prepared as input for the embedding model.

3. Embedding Model:

Sentence Transformers with the all-MiniLM-L6-v2 model converts each document into a numerical vector representation.

4. FAISS Index:

The generated vectors are stored in a FAISS IndexFlatL2 index. FAISS uses vector similarity to identify documents that are closest to the query vector.

5. Query Processing:

A user query is converted into an embedding using the same Sentence Transformer model.

6. Semantic Search:

The query vector is compared against the vectors stored in FAISS.

7. Retrieval:

The top relevant documents are retrieved according to vector similarity.

8. Benchmarking:

The system measures indexing time and average search latency for every dataset size.

9. Analysis:

The measured values are stored in a table and represented using graphs to analyze scalability.

Design Decisions

The following design decisions were made during implementation:

Selection of Sentence Transformer

The all-MiniLM-L6-v2 Sentence Transformer model was selected because it provides meaningful semantic embeddings while being lightweight enough for execution on a normal computer.

Selection of FAISS

FAISS was selected because the objective of Question 13 is to evaluate vector-search scalability. FAISS provides efficient similarity-search functionality over dense vectors and allows the indexing and search performance to be measured directly.

Selection of IndexFlatL2

IndexFlatL2 was selected because it provides a straightforward exact nearest-neighbor search implementation. It is simple to understand and suitable for demonstrating the effect of dataset size without introducing additional indexing parameters.

Selection of Dataset Sizes

The dataset sizes of 100, 500, 1,000, 5,000, and 10,000 documents were selected to observe how indexing and retrieval performance changes as the vector collection grows.

Search Latency Measurement

Multiple searches are performed instead of measuring only one search. The average latency is calculated to provide a more reliable measurement and reduce the effect of individual timing fluctuations.

Use of Matplotlib

Matplotlib was selected to visualize the relationship between dataset size and indexing/search performance.

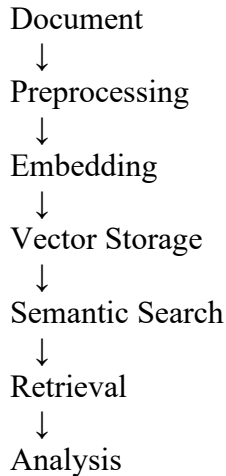
Methodology

The following methodology was followed:

1. Generate text documents for five different dataset sizes.
2. Preprocess the documents.
3. Generate embeddings using Sentence Transformers.
4. Measure the embedding-generation time.
5. Create a FAISS index.
6. Add the document vectors to the FAISS index.
7. Measure the indexing time.
8. Generate a query embedding.
9. Perform similarity search using FAISS.
10. Repeat the search multiple times.
11. Calculate average search latency.
12. Store the benchmark results.
13. Generate graphs.
14. Analyze FAISS scalability.

Complete Pipeline

The implementation follows the required pipeline:



Explanation

Document: Text documents are generated for different dataset sizes.

Preprocessing: The documents are converted into suitable text input for the embedding model.

Embedding: Sentence Transformer converts the documents into numerical vectors.

Vector Storage: The generated vectors are added to a FAISS index.

Semantic Search: The user query is also converted into an embedding.

Retrieval: FAISS searches for the most similar vectors.

Analysis: Indexing time and search latency are measured and compared.

Algorithm

Step 1

Start the program.

Step 2

Load the Sentence Transformer embedding model.

Step 3

Generate datasets containing:

- 100 documents
- 500 documents
- 1,000 documents
- 5,000 documents
- 10,000 documents

Step 4

Generate embeddings for each dataset.

Step 5

Create a FAISS index.

Step 6

Insert the embeddings into the FAISS index.

Step 7

Measure indexing time.

Step 8

Convert a search query into an embedding.

Step 9

Perform Top-K semantic search.

Step 10

Repeat the search multiple times and calculate average latency.

Step 11

Store the measurements in a table.

Step 12

Plot indexing time and search latency.

Step 13

Analyze the scalability.

Step 14

Stop the program.

Implementation

The complete solution was implemented using Python.

The major functions used are:

```
generate_documents()
```

```
build_faiss_index()
```

```
measure_search_latency()
```

```
main()
```

Important Code Snippet

Put only an important part of the code here:

```
embeddings = model.encode(  
    documents,  
    convert_to_numpy=True  
)
```

```
index = faiss.IndexFlatL2(  
    embeddings.shape[1]  
)
```

```
index.add(  
    embeddings.astype("float32")  
)
```

This code generates embeddings and stores them in the FAISS index.

Test Cases

Test Case	Dataset Size	Search Query
TC1	100	How is artificial intelligence used?
TC2	500	How is artificial intelligence used?
TC3	1,000	How is artificial intelligence used?
TC4	5,000	How is artificial intelligence used?
TC5	10,000	How is artificial intelligence used?

Execution Output

The program was executed successfully for all five dataset sizes.

```
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: D:/CSA6502-Gen AI/CO3_AT3_Q13_FAISS_Scalability_Benchmark.py =====

FAISS SCALABILITY BENCHMARK

Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 0% | 0/103 [00:00<?, ?it/s]Loading weights: 100% | 103/103 [00:00<00:00, 9082.02it/s]
Embedding model loaded successfully.

-----
Testing dataset: 100 documents
Documents generated: 100
Embedding time: 1.8239 seconds
FAISS indexing time: 0.001002 seconds
Vectors stored: 100
Average search latency: 0.107454 ms

-----
Testing dataset: 500 documents
Documents generated: 500
Embedding time: 4.4581 seconds
FAISS indexing time: 0.004166 seconds
Vectors stored: 500
Average search latency: 0.032246 ms

-----
Testing dataset: 1000 documents
Documents generated: 1000
Embedding time: 6.0241 seconds
FAISS indexing time: 0.001245 seconds
Vectors stored: 1000
Average search latency: 0.065277 ms

-----
Testing dataset: 5000 documents
Documents generated: 5000
Embedding time: 42.268 seconds
FAISS indexing time: 0.011051 seconds
Vectors stored: 5000
Average search latency: 2.975033 ms

-----
Testing dataset: 10000 documents
Documents generated: 10000
Embedding time: 116.8345 seconds
FAISS indexing time: 0.013131 seconds
Vectors stored: 10000
Average search latency: 1.572255 ms

=====
```

```
plt.title("FAISS Indexing Time vs Dataset Size")
plt.grid(True)

plt.savefig(
    "faiss_indexing_time.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

# =====
# GRAPH 2 - SEARCH LATENCY
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    results_df["Documents"],
    results_df["Average Search Latency (ms)"],
    marker="o"
)

plt.xlabel("Number of Documents")
plt.ylabel("Average Search Latency (ms)")
plt.title("FAISS Search Latency vs Dataset Size")
plt.grid(True)

plt.savefig(
    "faiss_search_latency.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print("\nBenchmark completed successfully.")

# =====
# PROGRAM START
# =====

if __name__ == "__main__":
    main()
```

```
Testing dataset: 100 documents
Documents generated: 100
Embedding time: 1.8239 seconds
FAISS indexing time: 0.001002 seconds
Vectors stored: 100
Average search latency: 0.107454 ms

-----
Testing dataset: 500 documents
Documents generated: 500
Embedding time: 4.4581 seconds
FAISS indexing time: 0.004166 seconds
Vectors stored: 500
Average search latency: 0.032246 ms

-----
Testing dataset: 1000 documents
Documents generated: 1000
Embedding time: 6.0241 seconds
FAISS indexing time: 0.001245 seconds
Vectors stored: 1000
Average search latency: 0.065277 ms

-----
Testing dataset: 5000 documents
Documents generated: 5000
Embedding time: 42.268 seconds
FAISS indexing time: 0.011051 seconds
Vectors stored: 5000
Average search latency: 2.975033 ms

-----
Testing dataset: 10000 documents
Documents generated: 10000
Embedding time: 116.8345 seconds
FAISS indexing time: 0.013131 seconds
Vectors stored: 10000
Average search latency: 1.572255 ms

=====
```

```
plt.title("FAISS Indexing Time vs Dataset Size")
plt.grid(True)

plt.savefig(
    "faiss_indexing_time.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

# =====
# GRAPH 2 - SEARCH LATENCY
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    results_df["Documents"],
    results_df["Average Search Latency (ms)"],
    marker="o"
)

plt.xlabel("Number of Documents")
plt.ylabel("Average Search Latency (ms)")
plt.title("FAISS Search Latency vs Dataset Size")
plt.grid(True)

plt.savefig(
    "faiss_search_latency.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print("\nBenchmark completed successfully.")

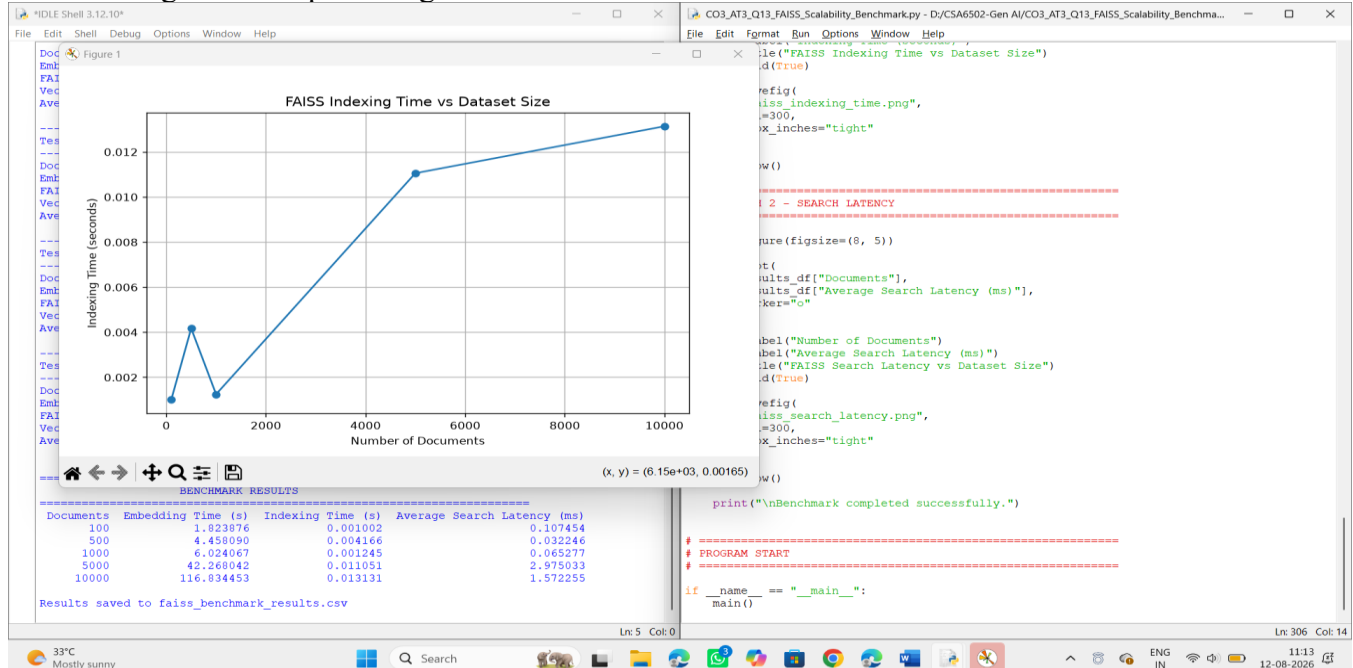
# =====
# PROGRAM START
# =====

if __name__ == "__main__":
    main()
```

The output displays the dataset size, embedding time, indexing time, number of stored vectors, and average search latency.

Graph – Indexing Time

The indexing time was plotted against the number of documents.



Analysis

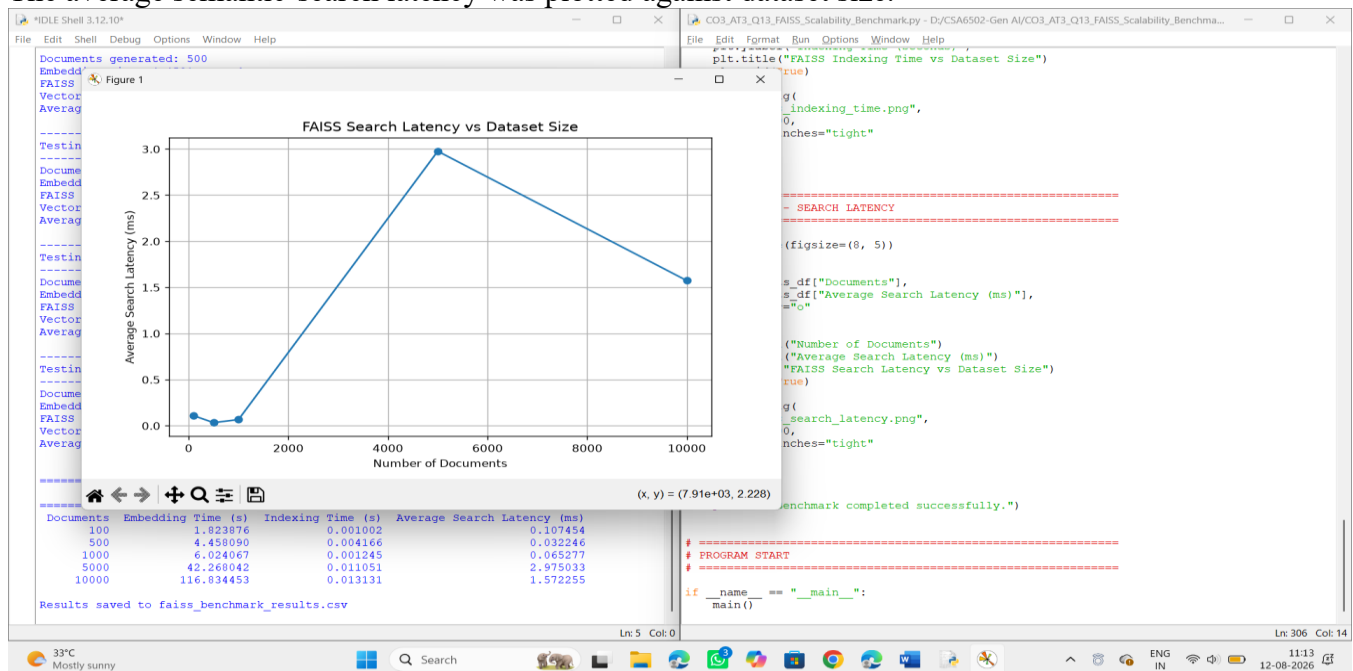
The graph shows the relationship between the number of documents and the time required to construct the FAISS index.

As the dataset size increases, more document embeddings have to be added to the index. Therefore, the indexing time generally increases with the number of documents.

Small variations in the measurements may occur because of CPU usage, memory availability, and other processes running on the computer.

Graph – Search Latency

The average semantic-search latency was plotted against dataset size.



Analysis

Search latency represents the average time required to perform similarity search for a query. The benchmark shows how retrieval performance changes as the number of stored vectors increases. The actual measurements may contain small fluctuations because search latency depends on system resources and execution conditions.

Semantic Retrieval Demonstration

Query

How is artificial intelligence used in modern applications?

The query is converted into an embedding and compared with the vectors stored in FAISS.

Retrieval Process

User Query



Query Embedding



FAISS Similarity Search



Top-5 Similar Vectors



Retrieved Documents

The retrieved documents are ranked according to their vector similarity with the query.

Scalability Analysis

The experiment demonstrates the effect of increasing dataset size on FAISS performance.

The major observations are:

1. The number of stored vectors increases with dataset size.
2. Indexing time generally increases as more vectors are inserted.
3. Search latency remains relatively low for the tested datasets.
4. Small timing fluctuations can occur because of system conditions.
5. FAISS provides efficient similarity search for dense embeddings.
6. The benchmark demonstrates the practical scalability of FAISS for the tested dataset sizes.

Result

The FAISS scalability benchmark was successfully implemented using Python and Sentence Transformers.

Datasets containing 100, 500, 1,000, 5,000, and 10,000 documents were processed. Embeddings were generated and stored in FAISS indexes.

Indexing time and average search latency were measured for every dataset size. The results were displayed using a benchmark table and graphs.

Conclusion

The experiment successfully demonstrated the scalability of FAISS for semantic vector search. The complete pipeline from document generation to preprocessing, embedding, vector storage, semantic search, retrieval, and analysis was implemented.

18. ChromaDB Persistence

Develop a persistent ChromaDB semantic-search application. Insert documents and embeddings, terminate the application, restart it, and perform retrieval. Demonstrate that the collection remains available and analyze why persistence is important.

Problem Statement

Develop a persistent ChromaDB semantic-search application. Insert documents and embeddings, terminate the application, restart it, and perform retrieval. Demonstrate that the collection remains available and analyze why persistence is important.

Aim

To develop a persistent semantic-search application using ChromaDB and Sentence Transformer embeddings and demonstrate that stored documents and embeddings remain available after application restart.

Objectives

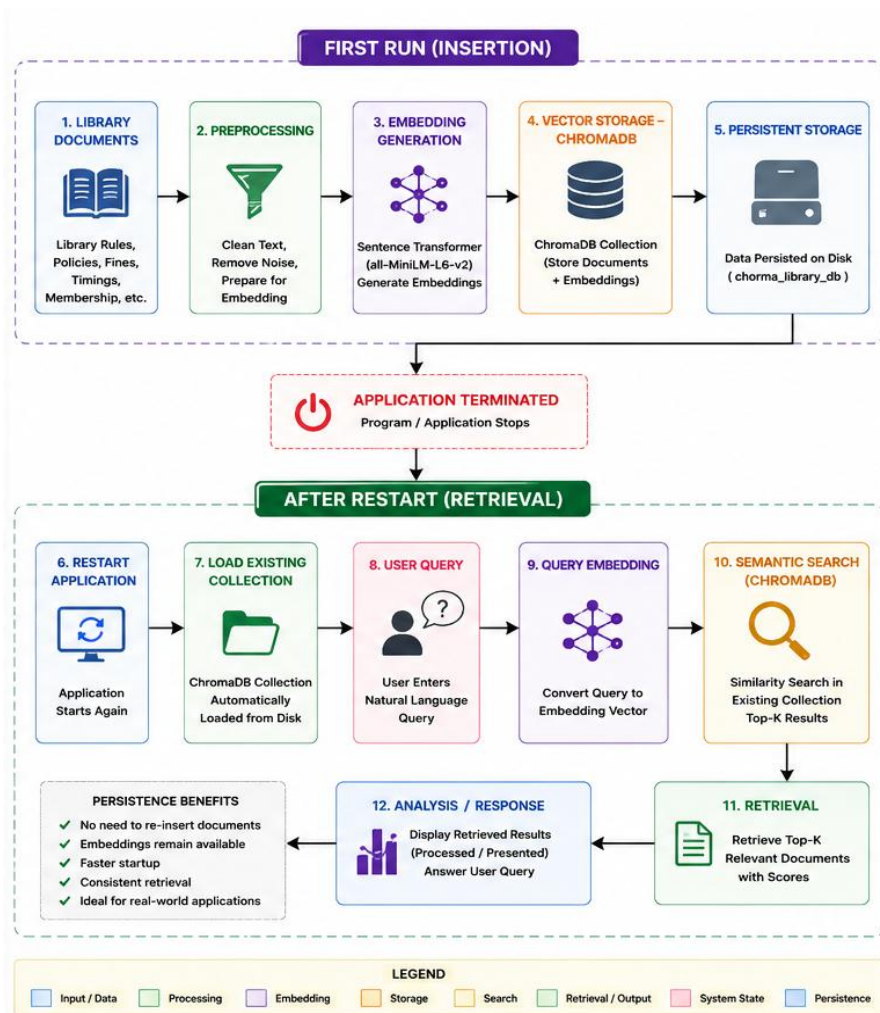
1. To create a library information document collection.
2. To preprocess the documents.
3. To generate embeddings using Sentence Transformers.
4. To store embeddings in ChromaDB.
5. To configure persistent storage.
6. To perform semantic search.
7. To terminate the application.
8. To restart the application.
9. To verify that the collection remains available.
10. To perform retrieval after restart.
11. To analyze the importance of persistence.

Software and Technologies Used

Technology	Purpose
Python	Program implementation
Sentence Transformers	Embedding generation
all-MiniLM-L6-v2	Embedding model
ChromaDB	Persistent vector database

System Architecture

The ChromaDB persistence application follows this architecture:



Architecture Explanation

The system contains the following components:

1. Library Documents:

The knowledge base contains library rules related to borrowing, renewal, fines, timings, membership, and other library information.

2. Preprocessing:

The documents are prepared as text entries suitable for embedding generation.

3. Embedding Model:

Sentence Transformer converts the library documents into numerical semantic vectors.

4. ChromaDB Collection:

The documents and their embeddings are stored in a ChromaDB collection.

5. Persistent Storage:

ChromaDB stores the collection in the specified persistent directory:
chroma_library_db

6. Application Restart:

When the application is terminated and restarted, the same persistent database path is opened again.

7. Collection Recovery:

The existing collection is detected instead of creating and inserting all documents again.

8. Query Processing:

The user's question is converted into an embedding.

9. Semantic Retrieval:

ChromaDB compares the query embedding with stored embeddings and returns the most relevant documents.

Design Decisions

Selection of Sentence Transformer

The all-MiniLM-L6-v2 model was selected because it generates semantic embeddings efficiently and is suitable for a small library knowledge base.

Selection of ChromaDB

ChromaDB was selected because Question 18 specifically focuses on persistence. ChromaDB provides a convenient persistent client that allows the vector collection to remain available between application executions.

Selection of Library Documents

Library rules were selected because they provide a simple and realistic semantic-search knowledge base. The documents contain information about borrowing, renewal, fines, timings, and membership.

Persistent Database Path

The following path was selected:

```
DB_PATH = "./chroma_library_db"
```

Using a fixed persistent path ensures that the application reconnects to the same database after restarting.

Avoiding Duplicate Insertion

The program checks the existing collection size before inserting documents:

```
if collection.count() == 0:
```

```
    # Insert documents
```

```
else:
```

```
    # Use existing collection
```

This design prevents the same documents from being unnecessarily inserted every time the application starts.

Top-K Retrieval

The application retrieves the top 3 most relevant documents. This provides enough context to demonstrate semantic retrieval while keeping the output readable.

Persistence Verification

The application is intentionally stopped and restarted. If the existing collection is detected and retrieval still works, it provides direct evidence that persistence is functioning correctly.

Library Knowledge Base

The system contains library-related information about:

- Book borrowing
- Loan duration
- Renewal
- Fines
- Library timings
- Membership
- Reference books
- Digital resources

Example documents include:

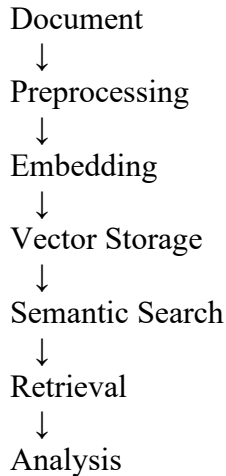
Students can borrow up to 4 books at a time.

Books can normally be borrowed for 14 days.

A fine of Rs. 2 per day is charged for each overdue book.

Complete Pipeline

The required pipeline is:



Explanation

Document: Library rules are used as the knowledge base.

Preprocessing: The documents are prepared as text inputs.

Embedding: Sentence Transformer converts documents into vectors.

Vector Storage: The vectors are stored in ChromaDB.

Semantic Search: User queries are converted into embeddings.

Retrieval: ChromaDB retrieves the most relevant documents.

Analysis: Persistence is verified by restarting the application and performing retrieval again.

Methodology

1. Load the library documents.
2. Load the Sentence Transformer model.
3. Generate document embeddings.
4. Create a persistent ChromaDB client.
5. Create or load the collection.
6. Insert documents if the collection does not already exist.
7. Accept user queries.
8. Generate query embeddings.
9. Perform semantic search.
10. Display retrieved documents.
11. Terminate the application.
12. Restart the application.
13. Reopen the same ChromaDB database.
14. Verify that the existing collection is available.
15. Perform retrieval again.
16. Analyze persistence.

Algorithm

Step 1

Start the program.

Step 2

Load the embedding model.

Step 3

Load the library documents.

Step 4

Generate document embeddings.

Step 5

Create a persistent ChromaDB client.

Step 6

Create or open the collection.

Step 7

Check whether documents already exist.

Step 8

Insert documents if the collection is empty.

Step 9

Accept a user query.

Step 10

Generate an embedding for the query.

Step 11

Perform semantic search.

Step 12

Display the retrieved documents.

Step 13

Terminate the program.

Step 14

Restart the program.

Step 15

Verify that the existing collection is available.

Step 16

Perform another semantic search.

Step 17

Display retrieved results.

Implementation

The major functions used are:

`create_embeddings()`

`get_collection()`

`insert_documents()`

`search_collection()`

`display_results()`

`main()`

Important Code Snippet

```
client = chromadb.PersistentClient(  
    path=DB_PATH  
)
```

```
collection = client.get_or_create_collection(  
    name=COLLECTION_NAME  
)
```

Test Cases

Test Case	Query	Expected Result
TC1	How many books can I borrow?	Borrowing rule
TC2	How long can I keep a book?	14-day loan period
TC3	Can I renew a book?	Renewal rule
TC4	What is the overdue fine?	Rs. 2/day
TC5	What are Saturday timings?	9 AM–5 PM
TC6	How can I become a member?	Membership information
TC7	What is today's hostel menu?	Unsupported information

First Execution – Document Insertion

The application was executed for the first time.

Input

How many books can I borrow?

Expected Retrieval

Students can borrow up to 4 books at a time.

```
CHROMADB PERSISTENCE DEMO
=====
Creating new persistence directory...
Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
o enable higher rate limits and faster downloads.
Loading weights: 0% | 0/103 [00:00<7, 7it/s]Loading weights: 100% | 103/103 [00:00<00:00, 10900.62it/s]
Embedding model loaded.
Connected to ChromaDB collection: library_documents
No existing documents found.
Adding documents to ChromaDB...
Documents inserted: 10
Current collection size: 10
Enter your library question: How many books can I borrow?
Performing semantic search...

=====
RETRIEVED DOCUMENTS
=====
Result 1
Distance: 0.5105
Document: Students can borrow up to 4 books at a time.
Result 2
Distance: 0.6204
Document: Books can normally be borrowed for 14 days.
Result 3
Distance: 0.8998
Document: A borrowed book can be renewed once for another 7 days if it has not been rese
rved by another user.
=====
ChromaDB persistence demonstration completed.
>>>
```

```
CO3_AT3_Q18_ChromaDB_Persistence.py - D:/CSA6502-Gen AI/CO3_AT3_Q18_ChromaDB_Persistence.py (3.12...)
File Edit Shell Debug Options Window Help
import os
import chromadb
from sentence_transformers import SentenceTransformer

# =====
# CONFIGURATION
# =====
DB_PATH = "./chroma_library_db"
COLLECTION_NAME = "library_documents"
EMBEDDING_MODEL = (
    "sentence-transformers/all-MiniLM-L6-v2"
)

# =====
# DOCUMENTS
# =====
DOCUMENTS = [
    "Students can borrow up to 4 books at a time.",
    "Books can normally be borrowed for 14 days.",
    "A borrowed book can be renewed once for another "
    "7 days if it has not been reserved by another user.",
    "A fine of Rs. 2 per day is charged for each "
    "overdue book.",
    "The library is open from 8:00 AM to 8:00 PM "
    "Monday to Friday.",
    "The library is open from 9:00 AM to 5:00 PM "
    "on Saturday.",
    "The library remains closed on Sundays and "
    "declared holidays.",
    "Currently enrolled students can become library "
```

Application Termination

After the first execution, the application was terminated.

The ChromaDB data remained stored in the persistent directory:

chroma_library_db

The application was then restarted using the same Python program.

Second Execution – Persistence Demonstration

After restarting, the application detected the existing ChromaDB collection.

Expected Output

Existing collection found.

Documents already stored: 10

No new insertion required.

Current collection size: 10

Retrieval After Restart

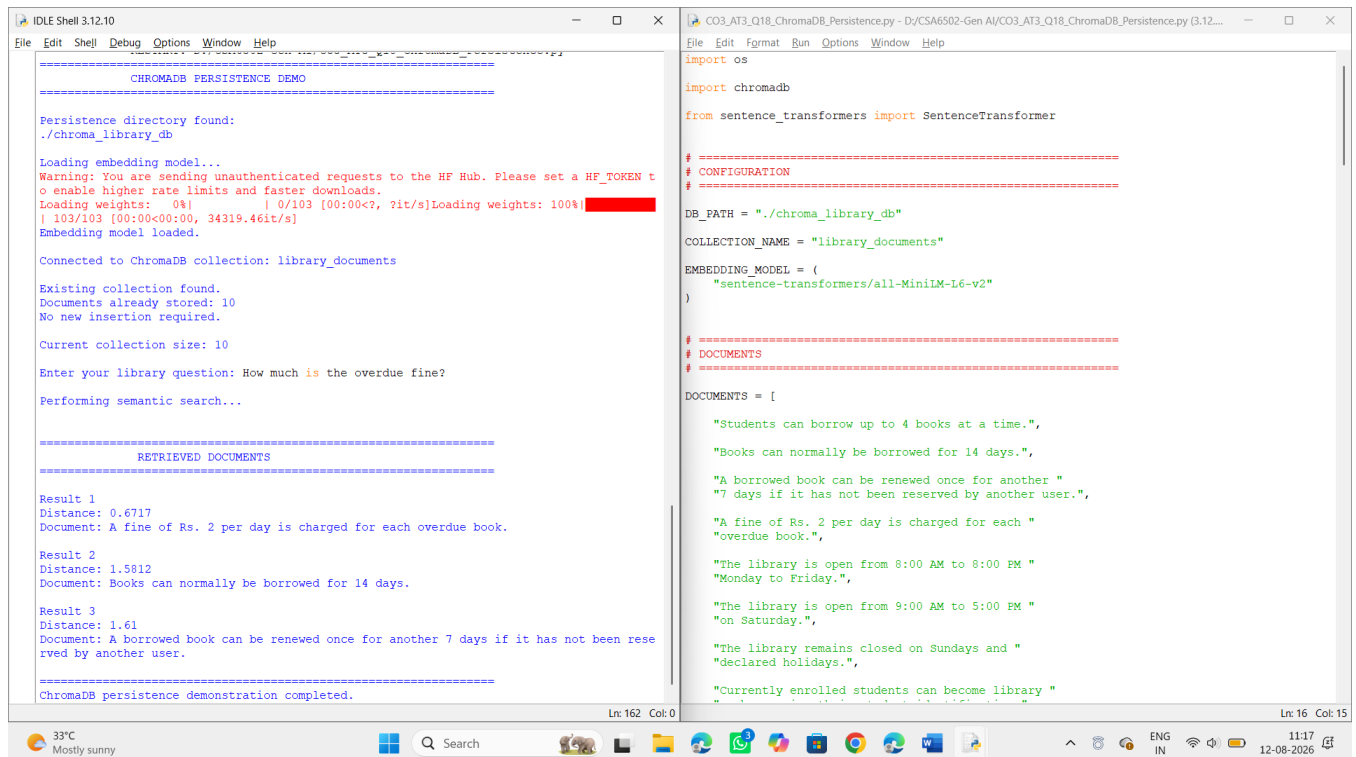
After restarting the application, the following query was tested:

Query

How much is the overdue fine?

Expected Retrieved Information

A fine of Rs. 2 per day is charged for each overdue book.



```
CHROMADB PERSISTENCE DEMO

Persistence directory found:
./chroma_library_db

Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 0% | 0/103 [00:00<?, ?it/s]Loading weights: 100% [103/103 [00:00<00:00, 34319.46it/s]
Embedding model loaded.

Connected to ChromaDB collection: library_documents

Existing collection found.
Documents already stored: 10
No new insertion required.

Current collection size: 10
Enter your library question: How much is the overdue fine?
Performing semantic search...

=====
RETRIEVED DOCUMENTS
=====

Result 1
Distance: 0.6717
Document: A fine of Rs. 2 per day is charged for each overdue book.

Result 2
Distance: 1.5812
Document: Books can normally be borrowed for 14 days.

Result 3
Distance: 1.61
Document: A borrowed book can be renewed once for another 7 days if it has not been reserved by another user.

=====
ChromaDB persistence demonstration completed.

import os
import chromadb
from sentence_transformers import SentenceTransformer

# =====
# CONFIGURATION
# =====

DB_PATH = "./chroma_library_db"
COLLECTION_NAME = "library_documents"

EMBEDDING_MODEL = (
    "sentence-transformers/all-MiniLM-L6-v2"
)

# =====
# DOCUMENTS
# =====

DOCUMENTS = [

    "Students can borrow up to 4 books at a time.",
    "Books can normally be borrowed for 14 days.",
    "A borrowed book can be renewed once for another "
    "7 days if it has not been reserved by another user.",
    "A fine of Rs. 2 per day is charged for each "
    "overdue book.",
    "The library is open from 8:00 AM to 8:00 PM "
    "Monday to Friday.",
    "The library is open from 9:00 AM to 5:00 PM "
    "on Saturday.",
    "The library remains closed on Sundays and "
    "declared holidays.",
    "Currently enrolled students can become library "
```

Additional Test Case – Library Timing

Query

What are the library timings on Saturday?

Expected Result

The system retrieves:

The library is open from 9:00 AM to 5:00 PM on Saturday.


```
===== RESTART: D:/CSA6502-Gen AI/CO3_AT3_Q18_ChromaDB_Persistence.py =====
CHROMADB PERSISTENCE DEMO

Persistence directory found:
./chroma_library_db

Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 0% | 0/103 [00:00<?, ?it/s]Loading weights: 100% [103/103 [00:00<00:00, 16939.04it/s]
Embedding model loaded.

Connected to ChromaDB collection: library_documents

Existing collection found.
Documents already stored: 10
No new insertion required.

Current collection size: 10

Enter your library question: What are Saturday timings?

Performing semantic search...

=====
RETRIEVED DOCUMENTS
=====

Result 1
Distance: 1.1237
Document: The library is open from 9:00 AM to 5:00 PM on Saturday.

Result 2
Distance: 1.1941
Document: The library is open from 8:00 AM to 8:00 PM Monday to Friday.

Result 3
Distance: 1.4904
Document: Books can normally be borrowed for 14 days.

=====
ChromaDB persistence demonstration completed.

Ln: 247 Col: 0

CO3_AT3_Q18_ChromaDB_Persistence.py - D:/CSA6502-Gen AI/CO3_AT3_Q18_ChromaDB_Persistence.py (3.12...)
File Edit Format Run Options Window Help

import os
import chromadb
from sentence_transformers import SentenceTransformer

# =====
# CONFIGURATION
# =====

DB_PATH = "./chroma_library_db"
COLLECTION_NAME = "library_documents"

EMBEDDING_MODEL = (
    "sentence-transformers/all-MiniLM-L6-v2"
)

# =====
# DOCUMENTS
# =====

DOCUMENTS = [

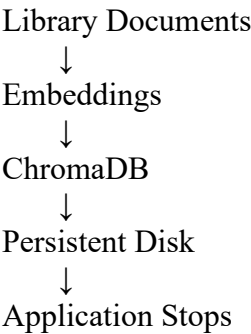
    "Students can borrow up to 4 books at a time.",
    "Books can normally be borrowed for 14 days.",
    "A borrowed book can be renewed once for another "
    "7 days if it has not been reserved by another user.",
    "A fine of Rs. 2 per day is charged for each "
    "overdue book.",
    "The library is open from 8:00 AM to 8:00 PM "
    "Monday to Friday.",
    "The library is open from 9:00 AM to 5:00 PM "
    "on Saturday.",
    "The library remains closed on Sundays and "
    "declared holidays.",
    "Currently enrolled students can become library "

]

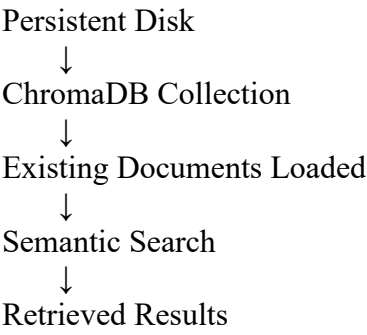
Ln: 29 Col: 45
```

Persistence Demonstration

The persistence process can be represented as:
FIRST EXECUTION



SECOND EXECUTION



This demonstrates that the data survives the termination and restart of the application.

Why Persistence Is Important

Persistence is important because vector databases can contain a large number of documents and their embeddings.

Without persistence, the application would need to:

Load documents



Generate embeddings



Insert vectors

every time the application starts.

With persistence, the stored collection can be reused:

Restart Application



Load Existing Collection



Perform Search

Therefore, persistence reduces unnecessary computation and improves the practicality of semantic-search applications.

Retrieval Relevance

The system uses semantic similarity rather than simple keyword matching.

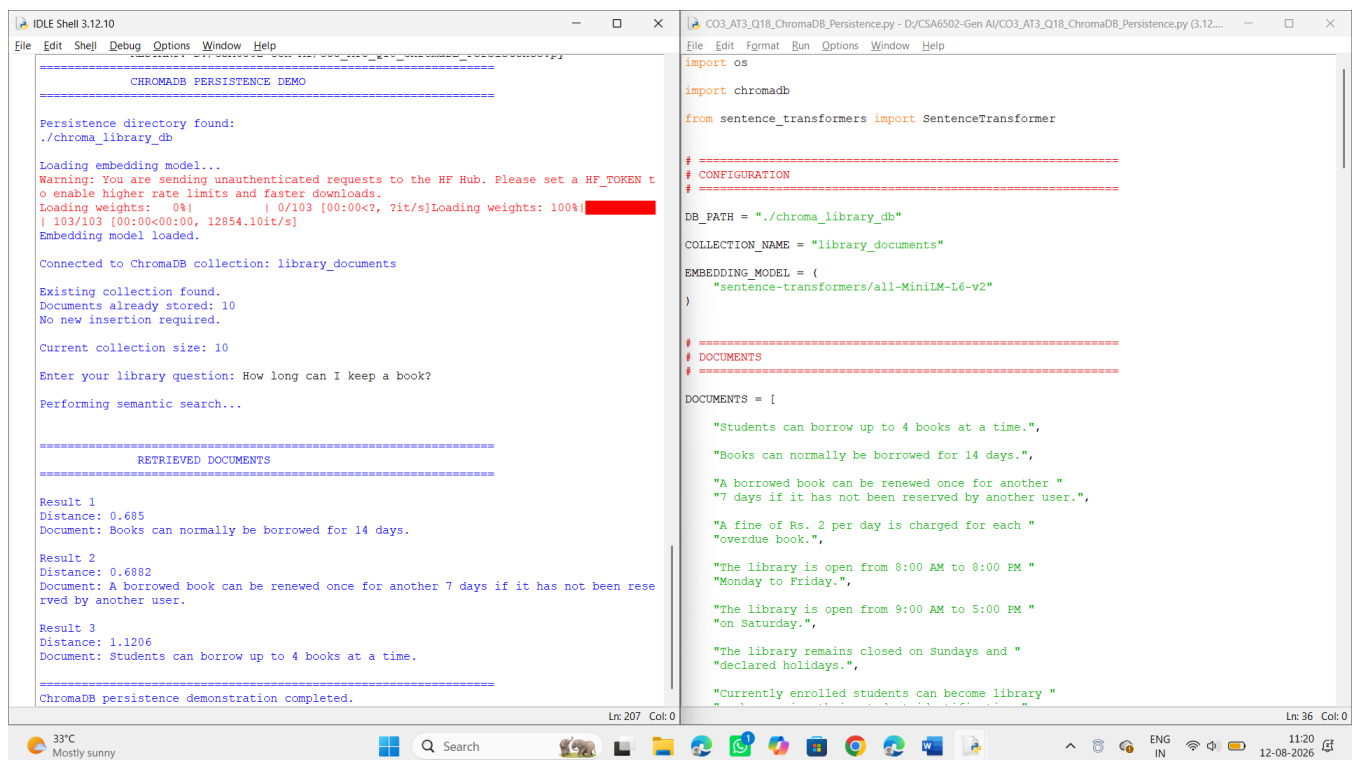
For example, the query:

How long can I keep a book?

can retrieve:

Books can normally be borrowed for 14 days.

The query does not necessarily need to contain the exact words present in the document because both the query and document are represented as semantic embeddings.



```
=====
CHROMADB PERSISTENCE DEMO
=====

Persistence directory found:
./chroma_library_db

Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
Loading weights: 0% | 0/103 [00:00<?, ?it/s]Loading weights: 100% ██████████
Embedding model loaded.

Connected to ChromaDB collection: library_documents

Existing collection found.
Documents already stored: 10
No new insertion required.

Current collection size: 10

Enter your library question: How long can I keep a book?

Performing semantic search...

=====
RETRIEVED DOCUMENTS
=====

Result 1
Distance: 0.685
Document: Books can normally be borrowed for 14 days.

Result 2
Distance: 0.6882
Document: A borrowed book can be renewed once for another 7 days if it has not been reserved by another user.

Result 3
Distance: 1.1206
Document: Students can borrow up to 4 books at a time.

=====
ChromaDB persistence demonstration completed.
=====
```

```
import os
import chromadb
from sentence_transformers import SentenceTransformer

# =====
# CONFIGURATION
# =====

DB_PATH = "./chroma_library_db"
COLLECTION_NAME = "library_documents"

EMBEDDING_MODEL = (
    "sentence-transformers/all-MiniLM-L6-v2"
)

# =====
# DOCUMENTS
# =====

DOCUMENTS = [

    "Students can borrow up to 4 books at a time.",
    "Books can normally be borrowed for 14 days.",
    "A borrowed book can be renewed once for another 7 days if it has not been reserved by another user.",
    "A fine of Rs. 2 per day is charged for each "
    "overdue book.",
    "The library is open from 8:00 AM to 8:00 PM "
    "Monday to Friday.",
    "The library is open from 9:00 AM to 5:00 PM "
    "on Saturday.",
    "The library remains closed on Sundays and "
    "declared holidays.",
    "Currently enrolled students can become library "
```

Result

The persistent ChromaDB semantic-search application was successfully implemented using Python and Sentence Transformers.

Library documents were converted into embeddings and stored in a persistent ChromaDB collection. The application was terminated and restarted, after which the existing collection was detected successfully.

Semantic queries were then performed after restarting the application, demonstrating that the stored information remained available.

Conclusion

The experiment successfully demonstrated persistent vector storage and semantic search using ChromaDB.

The complete pipeline:

Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis

was implemented successfully.

The restart test confirmed that the ChromaDB collection remained available after application termination. This demonstrates the importance of persistence for real-world semantic-search and RAG applications because previously generated embeddings do not have to be recreated every time the application starts.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Database Implementation	30%	Correctly builds and queries both FAISS and ChromaDB with accurate understanding of indexing	Builds both with minor issues	Only one database is built and queried correctly	Neither database functions correctly
Retrieval Relevance & Ranking	25%	Retrieval results are relevant and correctly ranked for all test queries	Mostly relevant results across queries	Inconsistent relevance across queries	Results are largely irrelevant
Comparative Analysis	20%	Thoughtful comparison of speed/accuracy trade-offs between the two databases	Reasonable comparison of the two databases	Superficial comparison with little insight	No meaningful comparison attempted
Benchmark Presentation	15%	Clear benchmark results presented (e.g. table or chart)	Results presented adequately	Results presented unclearly	No clear results presented
Methodology Documentation	10%	Clearly documents methodology and test queries used	Adequate documentation of methodology	Minimal documentation	No documentation of methodology

Marks Evaluation:

Question No.	Pipeline Functionality(30 %)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						